



GrabATable

Arquiteturas e Integração de Sistemas

Edgar Baptista, André Costa, João Ribeiro - Grupo 05
nrsº 16447, 21087, 23795

Orientação de
Bruno Lima

Data de entrega: 3 novembro

Mestrado em Engenharia Informática
Escola Superior de Tecnologia
Universidade Politécnica do Cávado e do Ave

Identificação dos alunos

Edgar Baptista, André Costa, João Ribeiro - Grupo 05
Números - 16447, 21087, 23795,
Mestrado em Engenharia Informática

Orientação

Bruno Lima

Índice

1 Introdução **1**

1.1 Âmbito do Projeto 1

1.1.1 Problema Identificado 1

1.1.2 Solução Proposta 1

1.1.3 Inovação 1

1.2 Público-Alvo e Tarefa Principal 1

1.3 Objetivos 2

1.4 Sistema Visual 3

1.4.1 Logótipo 3

1.4.2 Paleta Cromática 4

1.4.3 Tipografia 5

1.4.4 Grelha e Margens 5

1.4.5 Componentes Principais 5

1.4.6 Acessibilidade Cromática 6

2 Análise e Modelação do Sistema **7**

2.1 Site Map 7

2.2 Task Model 7

2.2.1 *Task Model* - Administrador 8

2.2.2 *Task Model* - Gestor 8

2.2.3 *Task Model* - Cliente 9

2.3 Requisitos Funcionais e Não Funcionais 9

2.3.1 Requisitos Funcionais 9

2.3.2 Requisitos Não Funcionais 10

2.4 Arquitetura Técnica do Sistema 11

2.5 BPMN 12

2.6 BPMN Atualizado 13

2.7 Personas 14

2.7.1 Persona 1 – Marta Silva (Cliente) 14
2.7.2 Persona 2 – Rui Fernandes (Cliente sem reserva) 15
2.7.3 Persona 3 – Teresa Almeida (Gestora) 16
2.7.4 Persona 4 – Pedro Machado (Administrador) 16
2.8 Análise SWOT 17

3 Protótipo Interativo 18
3.1 Acesso ao Protótipo 18
3.2 Fluxos de Interação 18
3.2.1 Administrador 18
3.2.2 Gestor 19
3.2.3 Cliente 21
3.3 Vídeo de Demonstração 22

4 Implementação da Arquitetura 23
4.1 Visão Geral e Tecnologias 23
4.2 API Gateway 23
4.3 Microsserviços 24
4.3.1 Serviço de Autenticação (Auth) 24
4.3.2 Serviço de Restaurantes (Restaurants) 24
4.3.3 Serviço de Reservas (Reservations) 25
4.3.4 Serviço de Fila de Espera (Waitlist) 26
4.3.5 Serviço de Notificações (Notifications) 26
4.4 Comunicação entre Serviços 27
4.5 Dados 27
4.6 Segurança 28
4.7 Documentação das APIs 28

5 Resiliência e Observabilidade 29
5.1 Princípios 29
5.2 Respostas de Erro 29
5.3 Resiliência na Comunicação Síncrona 30
5.4 Comportamento em Caso de Falhas 30
5.5 Limitações Conhecidas 31

6 Testes 32
6.1 Testes de Integração 32

6.2 Testes Obrigatórios 32
6.3 Resultados 33
6.4 Testes de Sistema 34

7 Containers – Docker **35**
7.1 Imagens 35
7.2 Ambiente com Docker Compose 35

8 Conclusão e Trabalho Futuro **36**
8.1 Requisitos Cumpridos 36
8.2 Conclusão 37
8.3 Trabalho Futuro 38

Bibliografia **39**

Lista de Figuras

1.1 Logótipo 3
1.2 Paleta Cromática 4

2.1 Site Map 7
2.2 Task Model - Administrador 8
2.3 Tarefa principal - Administrador 8
2.4 Task Model - Gestor 8
2.5 Tarefa principal - Gestor 8
2.6 Task Model - Cliente 9
2.7 Tarefa principal - Cliente 9
2.8 Arquitetura Técnica do Sistema 12
2.9 Diagrama BPMN 13
2.10 Diagrama BPMN atualizado 13

3.1 Administrador – Adicionar um restaurante (tarefa principal) 19
3.2 Administrador – Rever e bloquear um cliente (tarefa secundária) 19
3.3 Gestor – Atender um cliente com reserva (tarefa principal) 20
3.4 Gestor – Libertar uma mesa (tarefa secundária) 20
3.5 Cliente – Reservar mesa para refeição (tarefa principal) 21
3.6 Cliente – Entrar na fila de espera (tarefa secundária) 22

Lista de Tabelas

1.1 Público-alvo e tarefa principal 2
1.2 Paleta cromática 4
1.3 Grelha responsiva por tipo de dispositivo 5
1.4 Acessibilidade cromática 6

2.1 Análise SWOT do GrabATable 17

4.1 Eventos trocados entre serviços 27

5.1 Códigos de erro devolvidos à aplicação 30
5.2 Comportamento do sistema com cada peça parada 31

6.1 Testes obrigatórios 33
6.2 Testes unitários e de integração por serviço (todos aprovados) 33

7.1 Containers do ambiente 35

8.1 Rastreabilidade dos requisitos 37

1. Introdução

1.1 Âmbito do Projeto

GrabATable – Plataforma de gestão de reservas de restaurantes

A plataforma GrabATable consiste num sistema de gestão de reservas para restaurantes que combina dois mecanismos complementares: reservas antecipadas e gestão de fila de espera em tempo real para clientes sem reserva.

1.1.1 Problema Identificado

Os restaurantes que gerem reservas em papel, por telefone ou em sistemas isolados não conseguem conciliar as reservas com a afluência de clientes sem marcação. O resultado é sobrelotação, esperas imprevisíveis e uma má experiência tanto para quem reservou como para quem chegou sem reserva.

1.1.2 Solução Proposta

O cliente pode reservar mesa para uma data e hora, ou, ao chegar sem reserva, entrar numa fila virtual e acompanhar o tempo estimado. O sistema cruza reservas confirmadas, tempo médio de refeição e fila ativa para calcular a estimativa e avisa o cliente quando a mesa estiver quase disponível. O sistema cruza continuamente a disponibilidade de mesas (reservas confirmadas, tempo médio de refeição e fila de espera atual) para calcular estimativas dinâmicas, notificando o cliente quando a mesa estiver próxima de ficar disponível.

1.1.3 Inovação

A fila de espera não é uma lista à parte: é um recurso gerido em conjunto com as reservas, o que justifica uma arquitetura de microsserviços [1, 2] (cálculo de espera a partir de vários domínios, quase em tempo real).

1.2 Público-Alvo e Tarefa Principal

Há três perfis de utilizador, cada um com uma tarefa principal diferente.

Perfil	Quem é	Tarefa Principal	Contexto de uso
Cliente	Adultos dos 18 aos 55 anos, familiarizados com o uso do telemóvel	Reservar mesa ou entrar na fila e saber quanto tempo vai esperar	Telemóvel, muitas vezes a caminho do restaurante
Gestor	Gerente ou responsável de sala de um restaurante	Gerir reservas, mesas e fila durante o serviço	Tablet ou computador, sob pressão de tempo
Administrador	Equipa responsável pela plataforma	Configurar restaurantes, utilizadores e políticas globais	Computador, de forma pontual

Tabela 1.1: Público-alvo e tarefa principal

A tarefa central de todo o sistema é **garantir mesa sem surpresas:** o cliente sabe sempre se tem mesa e quando, e o gestor vê a ocupação e a fila num só ecrã.

1.3 Objetivos

O principal objetivo do projeto GrabATable é desenvolver uma plataforma de gestão de reservas e filas de espera para restaurantes, permitindo melhorar a organização da ocupação do estabelecimento e proporcionar aos clientes uma experiência mais previsível e eficiente.

A plataforma pretende centralizar a gestão de reservas antecipadas e de clientes sem reserva, utilizando a informação sobre a disponibilidade das mesas, reservas existentes, fila de espera e tempo médio de permanência para disponibilizar estimativas de espera atualizadas em tempo real.

Para concretizar o objetivo geral, o sistema deverá:

- Permitir aos clientes consultar a disponibilidade de mesas e efetuar reservas para uma data e hora específicas;
- Permitir aos clientes entrar numa fila de espera virtual quando não possuem reserva;
- Disponibilizar ao cliente uma estimativa do tempo de espera, atualizada de acordo com a ocupação e rotatividade do restaurante;
- Notificar o cliente quando a sua mesa estiver próxima de ficar disponível;
- Permitir aos gestores dos restaurantes visualizar e gerir as reservas existentes;
- Permitir aos gestores gerir a fila de espera em tempo real;
- Permitir registar a chegada dos clientes, bem como situações de falta de comparência;

- Permitir aos gestores configurar e atualizar o estado das mesas do restaurante;
- Apoiar a distribuição dos clientes pelas mesas disponíveis de forma organizada;
- Permitir aos administradores gerir os restaurantes registados na plataforma;
- Permitir a configuração de políticas gerais da plataforma, como regras de falta de comparência e parâmetros relacionados com o tempo médio de refeição;
- Centralizar a informação relacionada com reservas, mesas e filas de espera numa única plataforma;
- Garantir uma interface simples e intuitiva, adequada aos diferentes perfis de utilizador;
- Melhorar a previsibilidade dos tempos de espera e reduzir situações de sobrelotação ou má gestão da capacidade do restaurante.

1.4 Sistema Visual

A identidade combina um tom acolhedor, ligado à restauração, com interfaces limpas e legíveis.

1.4.1 Logótipo

Wordmark "GrabATable"em tipografia arredondada, com o "A"no centro. O logótipo apresenta uma identidade visual moderna, minimalista e acolhedora, associada ao setor da restauração e à reserva de mesas em restaurantes. Combina um símbolo gráfico facilmente reconhecível com um nome de marca de grande destaque, utilizando uma paleta de cores que transmite frescura, confiança e proximidade.



Figura 1.1: Logótipo

1.4.2 Paleta Cromática

A paleta cromática do GrabATable foi definida de forma a transmitir uma identidade visual simples, elegante e associada ao contexto de restauração. A paleta cromática foi gerada com a ferramenta Coolors [3].



Figura 1.2: Paleta Cromática

Papel	Nome	Hex	Uso
Primária	Verde floresta	#123F36	Texto principal, títulos, cabeçalhos
Secundária	Verde	#2A6B5C	Botões principais, ligações, estado confirmado
Destaque	Ouro	#C49A45	Fila de espera, ícones e realces
Apoio	Areia	#D6BB85	Etiquetas e fundos de realce, só com texto escuro
Superfície	Bege	#E8DCC4	Cartões, separadores e campos
Fundo	Creme	#F9F6F0	Fundo das páginas
Erro	Vermelho	#B91C1C	Erros e cancelamentos

Tabela 1.2: Paleta cromática

1.4.3 Tipografia

Títulos em *Poppins SemiBold* e texto corrido em Inter Regular, ambas gratuitas (Google *Fonts* [4]). Escala: 32, 24, 20 px para títulos; 16 px para texto; 14 e 12 px para apoio. Altura de linha 1,5 no texto corrido.

1.4.4 Grelha e Margens

Escala de espaçamento

- **4–8 px:** entre ícone e texto, ou dentro de componentes;
- **12–16 px:** *padding* de botões e cartões, e espaço entre campos de formulário;
- **24–32 px:** entre grupos de conteúdo;
- **48–80 px:** entre secções da página.

Grelha de colunas por tipo de dispositivo

Dispositivo	**Largura do ecrã**	**Colunas**	**Margem lateral**	***Gutter* (Espaço entre colunas)**
Telemóvel	< 600 px	4	16 px	16 px
Tablet	600-839 px	8	24 px	24 px
Computador	≥ 1200 px	12	32 px ou mais	24–32 px

Tabela 1.3: Grelha responsiva por tipo de dispositivo

1.4.5 Componentes Principais

- Botão primário, secundário e de texto, com estados normal, foco, premido e desativado;
- Cartão de restaurante (foto, nome, cozinha, disponibilidade);
- Seletor de data, hora e número de pessoas;
- Indicador de tempo de espera, com minutos e posição na fila;
- Etiqueta de estado da reserva (pendente, confirmada, cancelada, não comparência);
- Barra de navegação por perfil e menu lateral de gestão;
- Tabela de gestão com filtros e ações por linha;
- Notificações e alertas de erro em formulários.

1.4.6 Acessibilidade Cromática

O texto escuro sobre fundos claros e o texto claro sobre verde passam o nível AA da *WCAG* 2.1 [5] (mínimo 4,5:1 para texto normal); o ouro e a areia falham com texto claro ou verde. Os rácios foram calculados com a fórmula de luminância relativa da *WCAG*.

Texto / fundo	**Rácio**	**AA (4,5:1)**
Verde floresta / Creme	10,88	Passa
Verde floresta / Bege	8,64	Passa
Verde floresta / Areia	6,32	Passa
Vermelho / Creme	6,00	Passa
Verde / Creme	5,80	Passa
Verde / Bege	4,61	Passa
Verde floresta / Ouro	4,50	Passa, por margem mínima
Verde / Areia	3,37	Falha
Branco / Ouro	2,61	Falha
Creme / Ouro	2,42	Falha
Verde / Ouro	2,40	Falha
Ouro / Bege	1,92	Falha

Tabela 1.4: Acessibilidade cromática

Problema encontrado

A legenda do diagrama usa texto branco sobre amarelo (1,84), verde (2,28) e vermelho (3,76), e todas falham. Correção: texto Carvão sobre o amarelo (7,98) e sobre o verde, ou substituir os círculos por etiquetas com texto.

Regras adotadas

- O ouro nunca leva texto claro nem verde por cima; com ele usa-se Verde floresta (4,50:1, no limite);
- Ouro sobre Creme dá 2,42:1, abaixo dos 3:1 exigidos a ícones e bordas, pelo que estes usam Verde (5,80:1) e o ouro fica só como realce;
- A cor nunca é o único meio de informação: estados de reserva e fila têm sempre texto e ícone;
- Foco de teclado visível, com contorno de 2 px em Verde.

2. Análise e Modelação do Sistema

2.1 Site Map

O Site Map apresenta a estrutura e organização das principais páginas que constituem o sistema GrabATable, permitindo visualizar a hierarquia de navegação e a forma como as diferentes áreas e funcionalidades se encontram interligadas. Este permite também identificar os diferentes perfis de utilizador e as páginas e funcionalidades a que cada perfil terá acesso, proporcionando uma visão global da estrutura e navegação da plataforma.

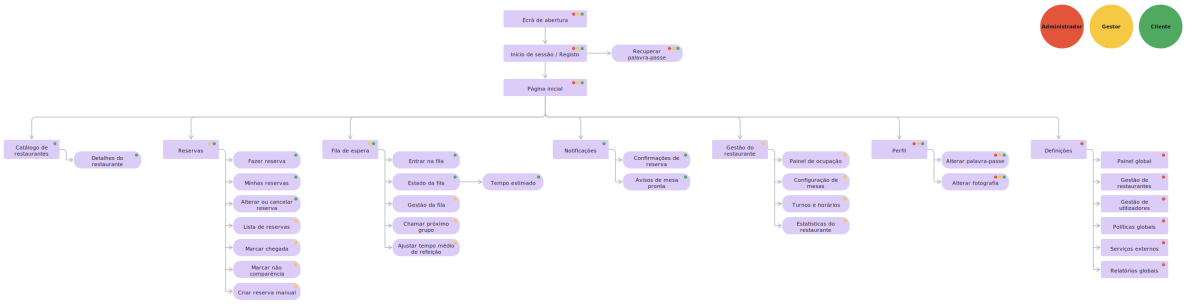


Figura 2.1: Site Map

Todos os perfis partilham o ecrã de abertura, o registo e a página inicial.

2.2 Task Model

O *Task Model* representa as principais tarefas que podem ser realizadas pelos diferentes perfis de utilizador no sistema GrabATable. Este modelo permite identificar e organizar as ações necessárias para a execução das principais funcionalidades da plataforma, evidenciando a relação entre os utilizadores e as tarefas que estes podem realizar.

2.2.1 *Task Model* - Administrador

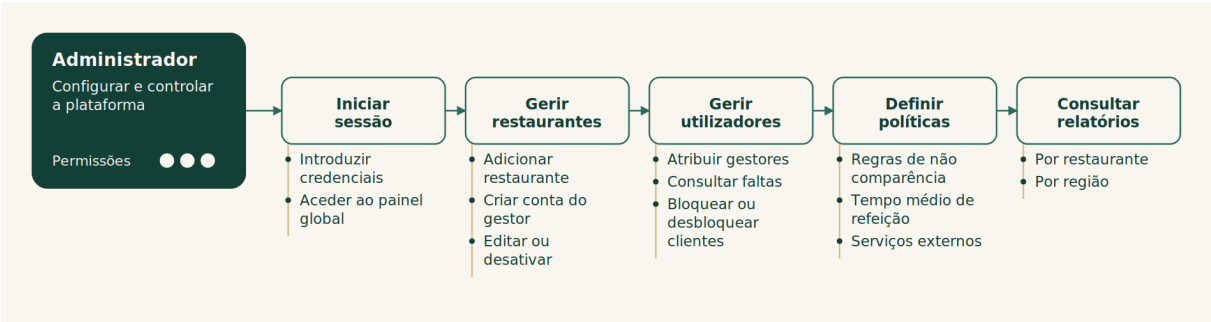


Figura 2.2: Task Model - Administrador

Tarefa principal - Administrador

Processo para um administrador adicionar um restaurante à plataforma.



Figura 2.3: Tarefa principal - Administrador

2.2.2 *Task Model* - Gestor



Figura 2.4: Task Model - Gestor

Tarefa principal - Gestor

Processo para um gestor atender um cliente com reserva.

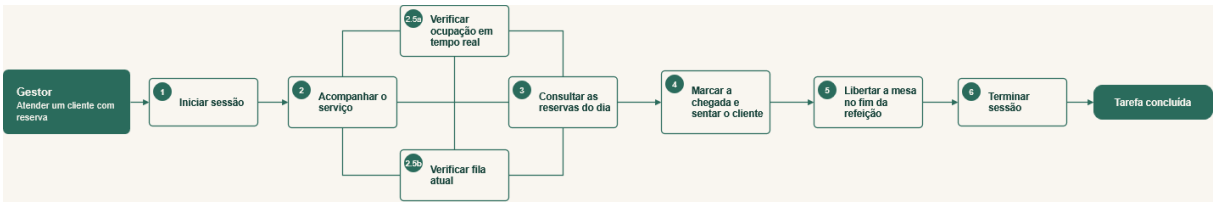


Figura 2.5: Tarefa principal - Gestor

2.2.3 *Task Model* - Cliente

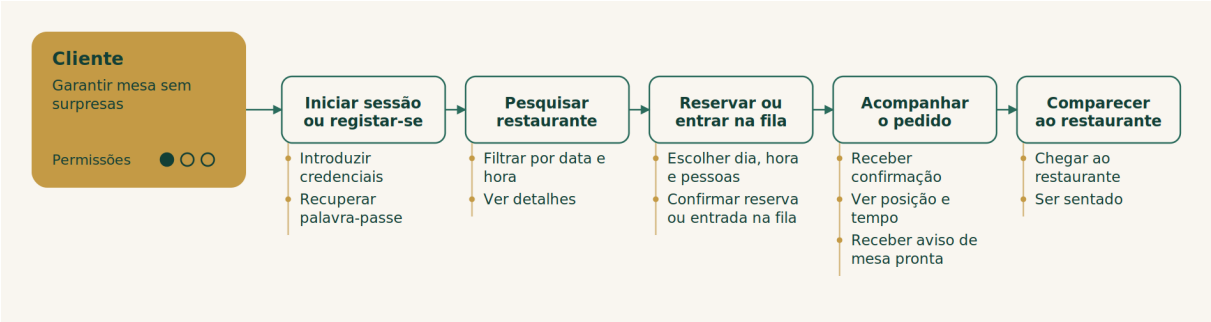


Figura 2.6: Task Model - Cliente

Tarefa principal - Cliente

Processo para um cliente reservar mesa para refeição.

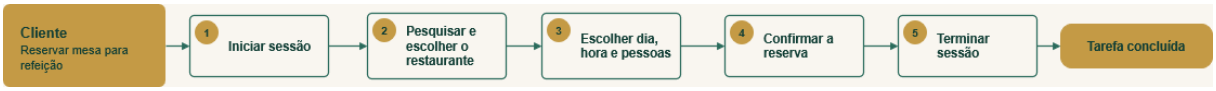


Figura 2.7: Tarefa principal - Cliente

2.3 Requisitos Funcionais e Não Funcionais

Nesta secção são apresentados os requisitos do sistema GrabATable, definidos a partir das funcionalidades e necessidades identificadas para a plataforma. Os requisitos encontram-se divididos em requisitos funcionais, que descrevem as funcionalidades e comportamentos que o sistema deve disponibilizar, e requisitos não funcionais, que especificam características relacionadas com a qualidade, desempenho, segurança, usabilidade do sistema, entre outros.

2.3.1 Requisitos Funcionais

O sistema GrabATable possui 17 requisitos funcionais, agrupados de acordo com os diferentes perfis de utilizador da plataforma.

- **RF01:** O sistema deve permitir que os utilizadores registem uma conta e iniciem sessão. **Prioridade:** Alta.
- **RF02:** O sistema deve permitir que os utilizadores recuperem a sua palavra-passe. **Prioridade:** Média.
- **RF03:** Os clientes devem poder pesquisar restaurantes por localização, tipo de cozinha, data e hora. **Prioridade:** Alta.

- **RF04:** Os clientes devem poder consultar os detalhes e a disponibilidade de um restaurante. **Prioridade:** Alta.

- **RF05:** Os clientes devem poder criar uma reserva indicando a data, hora e número de pessoas. **Prioridade:** Alta.

- **RF06:** Os clientes devem poder consultar, alterar e cancelar as suas reservas. **Prioridade:** Alta.

- **RF07:** Os clientes devem poder entrar na fila de espera e consultar a sua posição e o tempo estimado de espera. **Prioridade:** Alta.

- **RF08:** O sistema deve permitir que os clientes recebam notificações relativas à confirmação de uma reserva e à disponibilidade da mesa. **Prioridade:** Alta.

- **RF09:** Os gestores devem poder gerir as reservas do restaurante e registar a chegada ou não comparência dos clientes. **Prioridade:** Alta.

- **RF10:** Os gestores devem poder gerir a fila de espera, consultar a ordem dos clientes e chamar o próximo grupo. **Prioridade:** Alta.

- **RF11:** Os gestores devem poder configurar as mesas, respetivas capacidades, turnos e horários de funcionamento do restaurante. **Prioridade:** Alta.

- **RF12:** Os gestores devem poder consultar estatísticas relacionadas com a ocupação, não comparência e tempos de espera. **Prioridade:** Média.

- **RF13:** Os administradores devem poder criar, editar e remover restaurantes da plataforma. **Prioridade:** Alta.

- **RF14:** Os administradores devem poder gerir os utilizadores e bloquear clientes com um número excessivo de não comparências. **Prioridade:** Média.

- **RF15:** Os administradores devem poder definir políticas globais da plataforma, incluindo regras de não comparência e tempo médio de refeição. **Prioridade:** Média.

- **RF16:** Os administradores devem poder consultar relatórios globais organizados por restaurante e região. **Prioridade:** Baixa.

- **RF17:** O sistema deve permitir que os utilizadores alterem a sua palavra-passe e fotografia de perfil. **Prioridade:** Baixa.

2.3.2 Requisitos Não Funcionais

O sistema GrabATable possui 9 requisitos não funcionais, relacionados com características de qualidade e restrições técnicas da plataforma.

- **RNF01 – Desempenho:** As consultas devem apresentar uma resposta inferior a 500 ms no percentil 95. A estimativa de espera deve ser atualizada em menos de 5 segundos após uma alteração.

- **RNF02 – Segurança:** A autenticação deve utilizar *JWT* [6], validado no API *Gateway*, com comunicação através de HTTPS, palavras-passe armazenadas através de *hash* e controlo de acesso por perfil.

- **RNF03 – Segurança entre serviços:** Os microsserviços não devem ficar diretamente expostos a pedidos externos, sendo o API Gateway o único componente responsável pela receção de pedidos externos.

- **RNF04 – Resiliência:** Uma falha no serviço de Notificações não deve impedir a criação de reservas, sendo utilizadas mensagens assíncronas para a comunicação entre serviços (RabbitMQ [7]).

- **RNF05 – Escalabilidade:** Os microsserviços devem ser executados em containers Docker [8] e não guardar estado em memória: os dados ficam na respetiva base de dados PostgreSQL e as mensagens no RabbitMQ [7]. Desta forma, é possível executar várias instâncias do mesmo serviço sem alterar o código. [8].

- **RNF06 – Facilidade de Manutenção:** A arquitetura deve utilizar um microsserviço por área de negócio, com base de dados própria, migrações por versões e *APIs* documentadas através de *OpenAPI* [9].

- **RNF07 – Usabilidade e acessibilidade:** A plataforma deve seguir uma abordagem *mobile first* [10] e permitir a realização das ações principais em, no máximo, três toques [11].

- **RNF08 – Testabilidade:** Cada serviço deve possuir testes unitários, testes de integração e uma *collection* Postman [12] para validação das respetivas *API's*.

- **RNF09 – Privacidade:** A plataforma deve cumprir o *RGPD* [13], garantindo a utilização mínima de dados, a obtenção de consentimento quando aplicável e a possibilidade de eliminação da conta.

2.4 Arquitetura Técnica do Sistema

A arquitetura técnica do sistema apresenta a organização dos principais componentes tecnológicos que constituem a plataforma GrabATable e a forma como estes comunicam entre si. Esta representação permite compreender a distribuição das diferentes camadas do sistema.

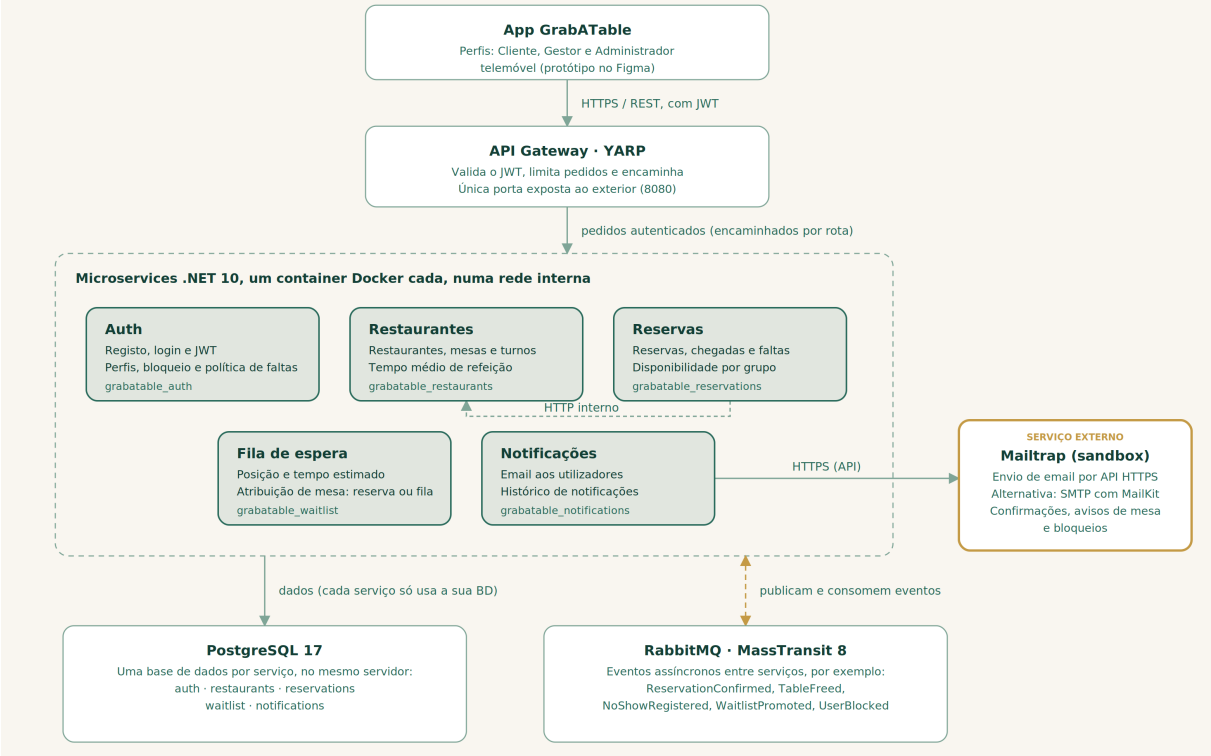


Figura 2.8: Arquitetura Técnica do Sistema

O *gateway* é o único ponto exposto: valida o *JWT* e encaminha cada pedido para o microsserviço certo. Cada serviço tem a sua base de dados no mesmo PostgreSQL [14]. Reservas e Fila de espera publicam eventos no *RabbitMQ* [7], e o serviço de Notificações consome-os para avisar o cliente.

O serviço de Notificações envia os emails através da API da sandbox do Mailtrap [15], por HTTPS. Em alternativa, pode usar qualquer servidor SMTP, através da biblioteca MailKit [16]. Sem nenhum dos dois configurado, as mensagens ficam apenas registadas no log, o que permite desenvolver e testar sem enviar emails.

2.5 BPMN

O diagrama BPMN [17] representa o fluxo dos principais processos do sistema GrabATable, permitindo visualizar de forma estruturada as atividades realizadas pelos diferentes intervenientes. Através deste modelo, é possível compreender a sequência das tarefas, as decisões e interações entre os diferentes participantes, proporcionando uma visão global do funcionamento dos processos da plataforma.

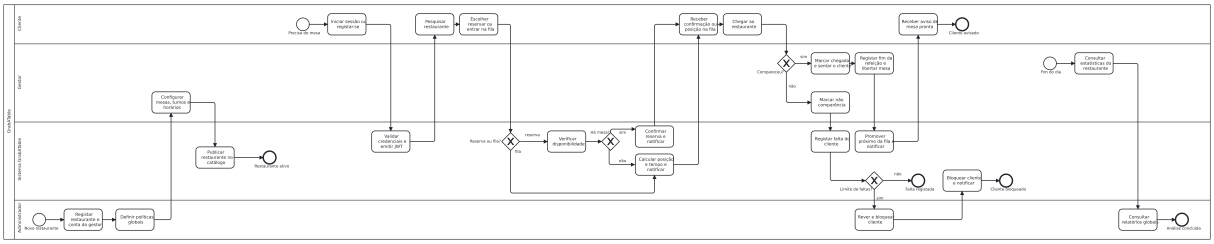


Figura 2.9: Diagrama BPMN

2.6 BPMN Atualizado

Após o feedback recebido relativamente à primeira entrega, procedemos à reformulação do diagrama BPMN [17]. A nova versão passou a considerar a existência de reservas próximas, a dimensão dos grupos e a implementação de uma janela de proteção para as reservas futuras. Foram também detalhadas as diferentes situações possíveis, incluindo a atribuição da mesa, a proteção de uma reserva próxima e a expiração de grupos em espera. Estas alterações visam dar resposta aos problemas identificados pelo professor.

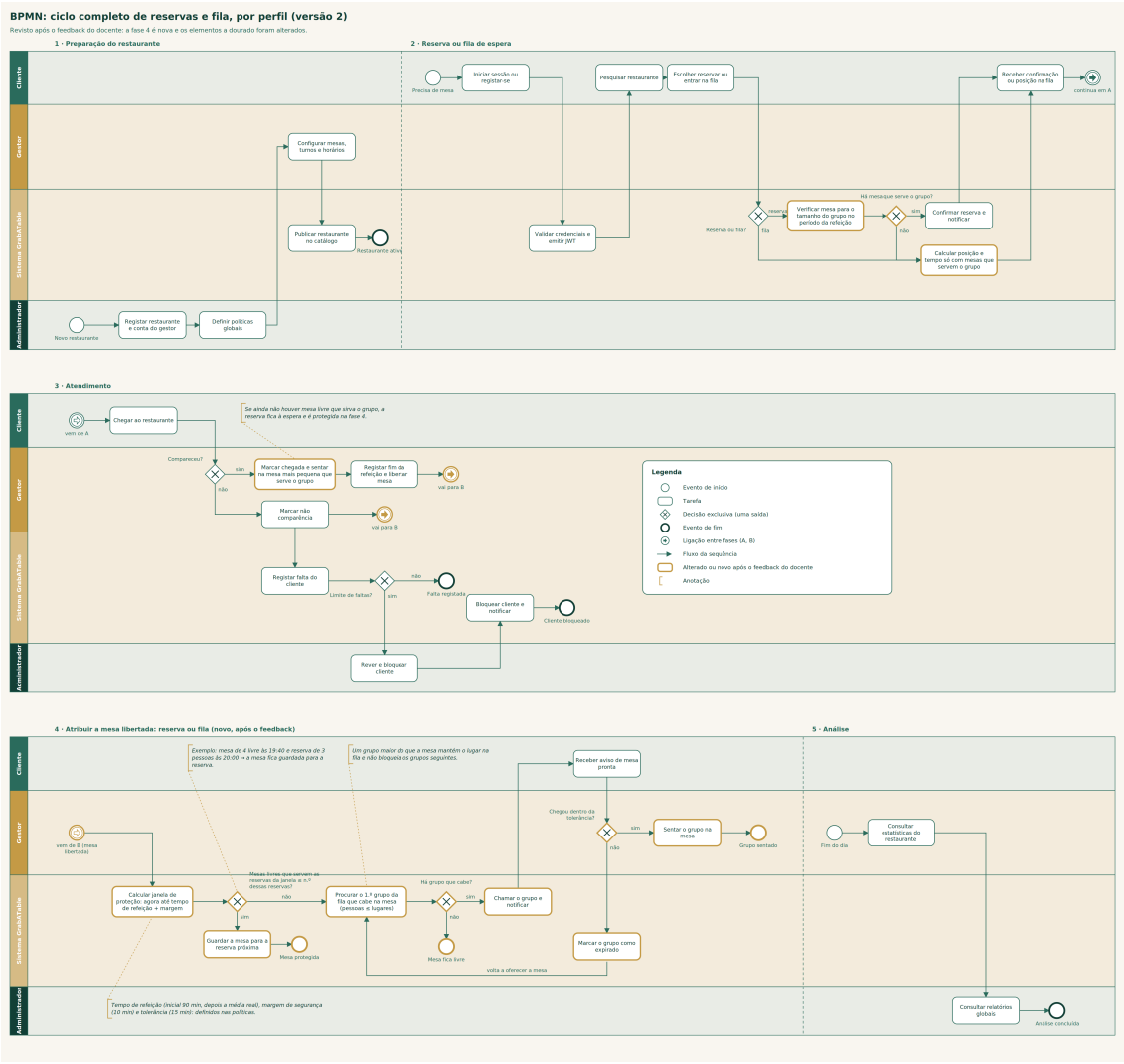


Figura 2.10: Diagrama BPMN atualizado

2.7 Personas

As *personas* são perfis fictícios, construídos a partir dos três perfis de utilizador definidos na Secção 1.2, que representam os objetivos, o contexto de uso e as frustrações dos utilizadores reais da plataforma.

2.7.1 Persona 1 – Marta Silva (Cliente)

Perfil	Cliente
Idade e profissão	34 anos, designer gráfica
Localização	Braga
Competência digital	Alta; usa o telemóvel para quase tudo, incluindo pagamentos e marcações
Contexto de uso	Telemóvel, em casa ou no trabalho, a organizar jantares com antecedência
Objetivos	• Reservar mesa para um grupo de 4 a 6 pessoas, sem telefonar; • Ter confirmação imediata e poder alterar ou cancelar a reserva; • Receber um lembrete antes da data.
Frustrações	• Restaurantes que só aceitam reservas por telefone, em horário de expediente; • Chegar à hora marcada e esperar por uma mesa que devia estar livre; • Não saber se a reserva ficou realmente registada.
Cenário	A Marta quer jantar com amigos na sexta-feira. Pesquisa restaurantes por cozinha e hora, confirma a disponibilidade para seis pessoas, reserva em poucos toques e recebe a notificação de confirmação.
Requisitos associados	RF03, RF04, RF05, RF06, RF08

2.7.2 Persona 2 – Rui Fernandes (Cliente sem reserva)

Perfil	Cliente
Idade e profissão	27 anos, técnico de logística
Localização	Guimarães
Competência digital	Média-alta; usa aplicações móveis no dia a dia, mas pouco paciente com processos longos
Contexto de uso	Telemóvel, muitas vezes a caminho do restaurante, com pouco tempo e uma só mão livre
Objetivos	• Entrar na fila antes de chegar ao restaurante; • Saber a posição e o tempo estimado de espera; • Ser avisado quando a mesa estiver quase pronta, para poder fazer outra coisa entretanto.
Frustrações	• Esperas de pé, sem qualquer estimativa; • Estimativas dadas ao balcão que nunca se cumprem; • Perder a vez por ter saído para ir buscar o carro.
Cenário	O Rui decide jantar fora de improviso. Abre a aplicação, escolhe um restaurante, entra na fila para duas pessoas e vê que a espera é de cerca de 25 minutos. Recebe um aviso quando a mesa está próxima de ficar disponível.
Requisitos associados	RF03, RF04, RF07, RF08

2.7.3 Persona 3 – Teresa Almeida (Gestora)

Perfil	Gestor
Idade e profissão	45 anos, responsável de sala num restaurante com 18 mesas
Localização	Porto
Competência digital	Média; usa tablet e computador, mas valoriza ferramentas simples
Contexto de uso	Tablet à entrada da sala, durante o serviço, sob pressão de tempo e com interrupções constantes
Objetivos	• Ver num só ecrã a ocupação das mesas, as reservas do dia e a fila; • Registar chegadas, sentar clientes e libertar mesas com poucos toques; • Reduzir as faltas de comparação e a sobrelotação.
Frustrações	• Caderno de reservas em papel e lista de espera improvisada; • Mesas reservadas que ficam vazias por não comparência; • Clientes sem reserva que se queixam de esperas sem explicação.
Cenário	Em plena hora de ponta, a Teresa regista a chegada de um grupo com reserva, vê que há uma mesa a libertar-se e chama o próximo grupo da fila, sem sair do tablet.
Requisitos associados	RF09, RF10, RF11, RF12

2.7.4 Persona 4 – Pedro Machado (Administrador)

Perfil	Administrador
Idade e profissão	38 anos, membro da equipa responsável pela plataforma
Localização	Remoto
Competência digital	Muito alta; confortável com painéis de administração e relatórios
Contexto de uso	Computador, de forma pontual, para configurar e supervisionar
Objetivos	• Adicionar e configurar novos restaurantes e respetivas contas de gestor; • Definir políticas globais (regras de não comparência, tempo médio de refeição); • Consultar relatórios por restaurante e por região.
Frustrações	• Configurações dispersas por vários ecrãs; • Falta de histórico sobre quem alterou o quê; • Dificuldade em identificar clientes com faltas repetidas.
Cenário	O Pedro integra um novo restaurante: cria o registo, cria a conta do gestor e confirma a configuração, tudo a partir da Gestão de restaurantes.
Requisitos associados	RF13, RF14, RF15, RF16

2.8 Análise SWOT

A análise SWOT identifica os fatores internos (forças e fraquezas) e externos (oportunidades e ameaças) que podem influenciar o sucesso do GrabATable. Serve para orientar prioridades de desenvolvimento e antecipar riscos.

Forças (interno)	**Fraquezas (interno)**
• Fila de espera gerida em conjunto com as reservas, e não como lista à parte; • Estimativa de espera dinâmica, calculada com reservas, tempo médio de refeição e fila ativa; • Arquitetura de microsserviços, resiliente e escalável (RNF04, RNF05); • Sistema visual com acessibilidade cromática verificada (WCAG 2.1 AA).	• A qualidade da estimativa depende de dados que o gestor tem de registar (chegadas, mesas libertadas); • Tempo médio de refeição é uma média, que pode falhar em casos atípicos;
Oportunidades (externo)	**Ameaças (externo)**
• Muitos restaurantes ainda gerem reservas em papel ou por telefone; • Elevada utilização do telemóvel e habituação a serviços de marcação digitais; • Valor acrescentado das estatísticas de ocupação e não comparência para os restaurantes; • Possibilidade de integração futura com sistemas de ponto de venda; • Expansão para outras regiões, aproveitando a escalabilidade horizontal.	• Plataformas de reservas já estabelecidas, com muitos restaurantes e utilizadores; • Resistência dos gestores a mudar processos durante o serviço; • Cumprimento do RGPD [13] e riscos associados a dados pessoais (RNF09); • Dependência de serviços externos de notificação e respetivos custos; • Não comparência e abuso da fila, que podem prejudicar a confiança no sistema.

Tabela 2.1: Análise SWOT do GrabATable

3. Protótipo Interativo

O protótipo interativo do *GrabATable* foi desenvolvido no Figma [18] e traduz o sistema visual definido na secção 1.4 (paleta, tipografia e componentes) em ecrãs navegáveis para os três perfis de utilizador. Para realizar os testes foi utilizado um iphone 13 pro.

3.1 Acesso ao Protótipo

O protótipo pode ser consultado e navegado através da seguinte ligação:

Abrir o protótipo no Figma

3.2 Fluxos de Interação

Para cada perfil são apresentados dois fluxos: uma **tarefa principal**, correspondente à tarefa específica modelada na secção 2.2 (*Task Model*), e uma **tarefa secundária**, que mostra uma funcionalidade complementar do mesmo perfil. Cada figura mostra, ecrã a ecrã, a sequência de interações, com os toques assinalados a dourado e uma legenda por passo.

3.2.1 Administrador

Tarefa principal – Adicionar um restaurante.

Corresponde ao processo de adicionar um restaurante à plataforma que foi desenhado no *Task Model* principal do administrador. O administrador inicia sessão, abre o separador Restaurantes, preenche os dados do restaurante e cria a conta do gestor em dois passos, e confirma a publicação no catálogo (RF13).

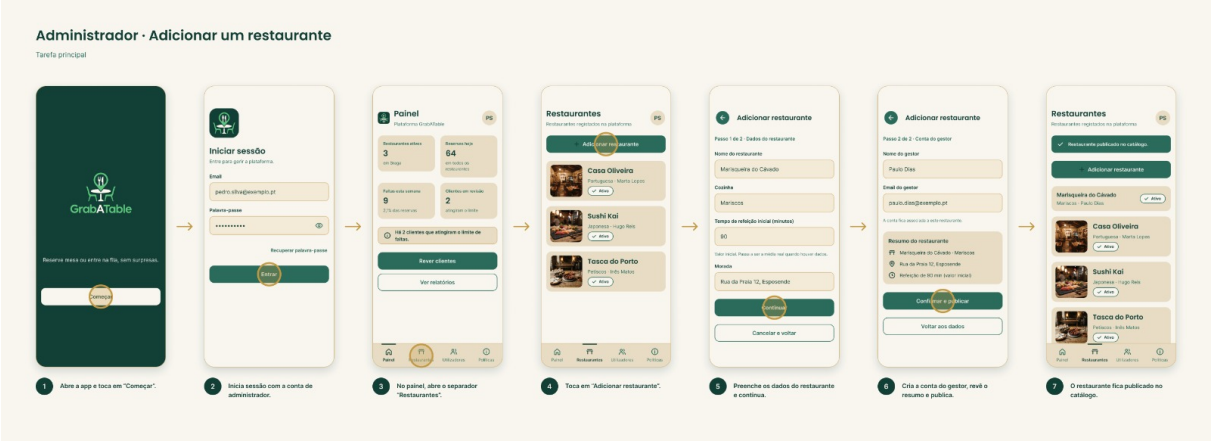


Figura 3.1: Administrador – Adicionar um restaurante (tarefa principal)

Tarefa secundária – Rever e bloquear um cliente.

A partir do aviso no painel, o administrador abre a lista de clientes *Em revisão*, consulta as faltas do cliente e confirma o bloqueio, sendo enviado um email ao cliente (RF14). A ação é reversível através do botão *Desbloquear cliente*.

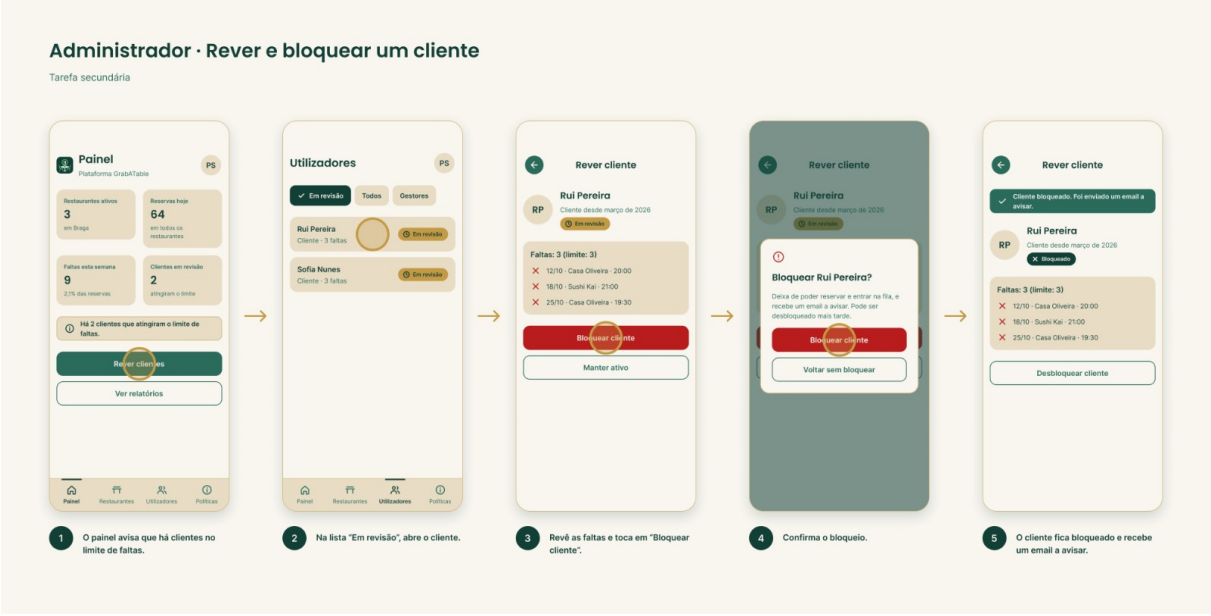


Figura 3.2: Administrador – Rever e bloquear um cliente (tarefa secundária)

3.2.2 Gestor

Tarefa principal – Atender um cliente com reserva.

Corresponde ao processo de atender um cliente com reserva que foi desenhado no *Task Model* principal do gestor. No painel de serviço, o gestor vê a próxima chegada, marca a chegada do cliente e confirma a mesa sugerida pelo sistema, que passa a ficar ocupada (RF09).



Figura 3.3: Gestor – Atender um cliente com reserva (tarefa principal)

Tarefa secundária – Libertar uma mesa.

Quando o grupo sai, o gestor abre o separador *Mesas* e liberta a mesa com dois toques. O sistema avisa automaticamente o primeiro grupo da fila de espera que cabe nessa mesa (RF10), o que mantém a estimativa de espera atualizada.

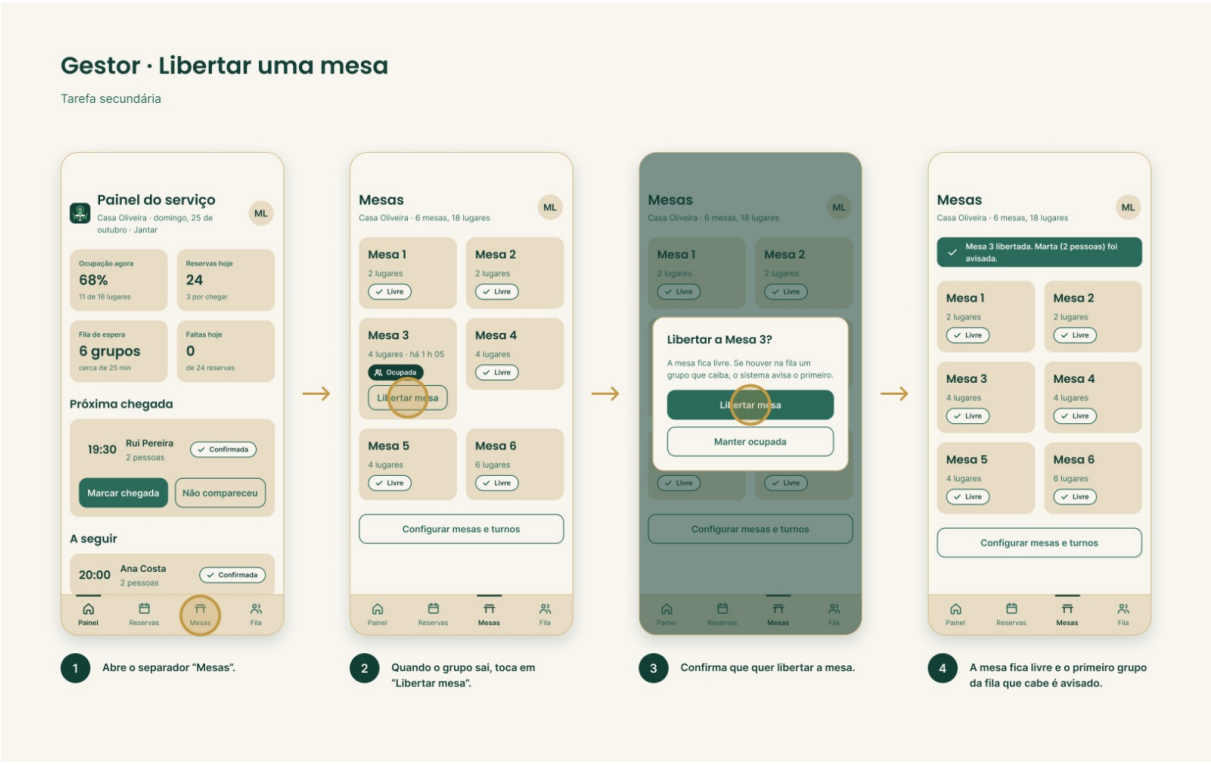


Figura 3.4: Gestor – Libertar uma mesa (tarefa secundária)

3.2.3 Cliente

Tarefa principal – Reservar mesa para refeição.

Corresponde ao processo de reservar mesa para uma refeição que foi desenhado no *Task Model* principal do cliente. O cliente inicia sessão, escolhe o restaurante, seleciona o dia, a hora e o número de pessoas e recebe a confirmação da reserva por email (RF03, RF04, RF05 e RF08).

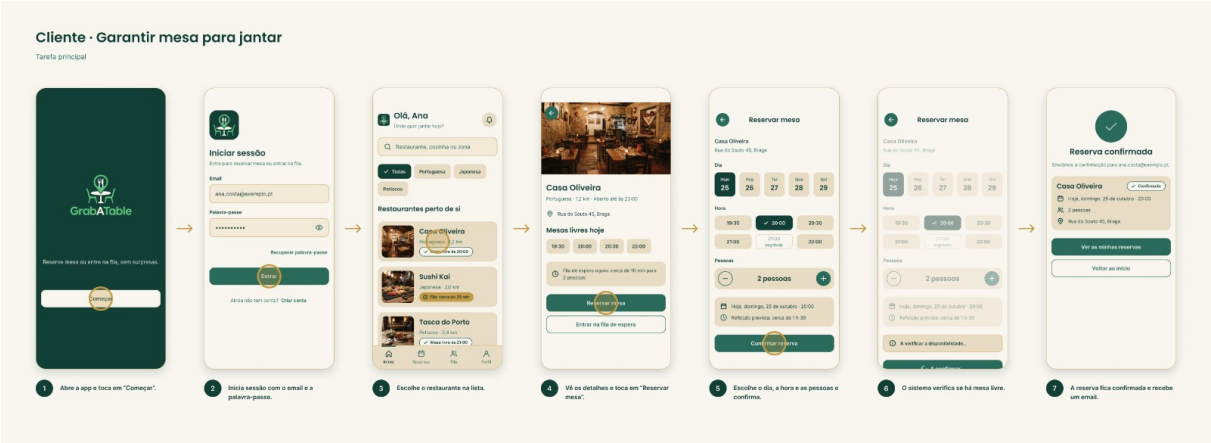


Figura 3.5: Cliente – Reservar mesa para refeição (tarefa principal)

Tarefa secundária – Entrar na fila de espera.

Para o cliente sem reserva, o fluxo permite entrar na fila, ver o tempo estimado, acompanhar a posição em tempo real e receber o aviso de que a mesa está quase pronta (RF07 e RF08).

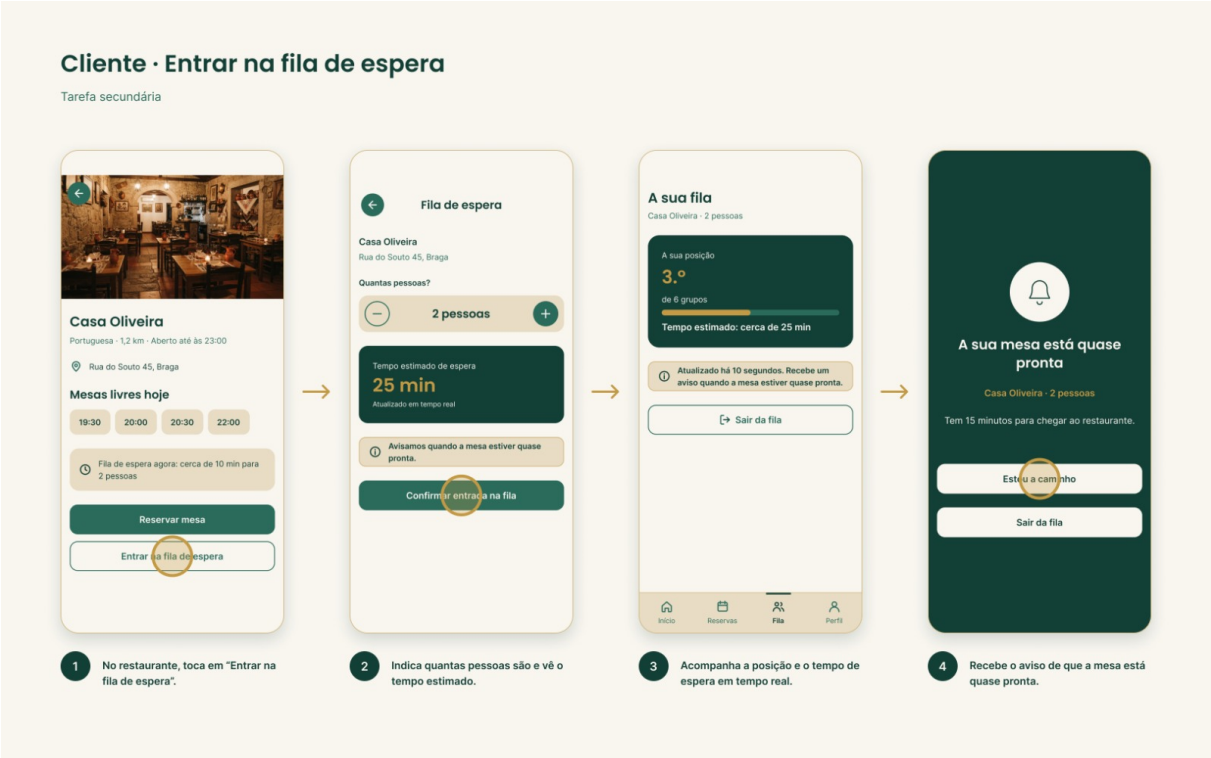
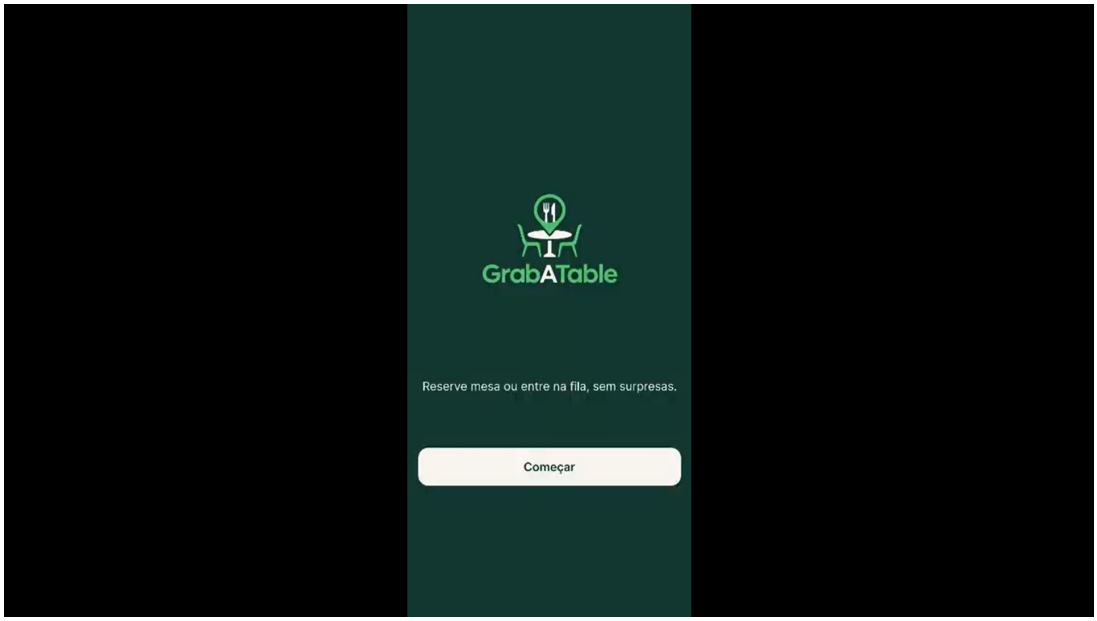


Figura 3.6: Cliente – Entrar na fila de espera (tarefa secundária)

3.3 Vídeo de Demonstração

Foi gravado um vídeo que demonstra a execução completa da tarefa principal definida para a aplicação. O vídeo está disponível em: *(clicar na imagem em baixo)*



4. Implementação da Arquitetura

Este capítulo descreve a solução implementada a partir da análise do Capítulo 2: as tecnologias escolhidas, o API Gateway, cada um dos cinco microsserviços, a forma como comunicam entre si, as regras de negócio e a segurança.

4.1 Visão Geral e Tecnologias

A plataforma é composta por um API Gateway e cinco microsserviços, um por área de negócio [1], cada um com a sua própria base de dados (RNF06). Os serviços comunicam sobretudo por eventos assíncronos através de um *broker* [7, 19] de mensagens; a única chamada síncrona entre serviços é a do serviço de reservas ao serviço de restaurantes, para obter os dados do restaurante no momento da reserva.

```
App (Cliente, Gestor, Administrador)
  -> API Gateway (YARP, porta 8080: valida o JWT, limita pedidos, encaminha)
       -> Auth           -> PostgreSQL: grabatable_auth
       -> Restaurants    -> PostgreSQL: grabatable_restaurants
       -> Reservations   -> PostgreSQL: grabatable_reservations (HTTP -> Restaurants)
       -> Waitlist       -> PostgreSQL: grabatable_waitlist
       -> Notifications  -> PostgreSQL: grabatable_notifications -> email (SMTP)
  Serviços <-> RabbitMQ (MassTransit): eventos entre serviços
  Gateway e serviços -> Jaeger (OpenTelemetry): rastreio de cada pedido
```

4.2 API Gateway

O gateway é o único componente exposto ao exterior (RNF03). As suas responsabilidades são:

- **Encaminhamento:** cada prefixo de rota (`/api/auth`, `/api/restaurants`, `/api/reservations`, `/api/waitlist`, `/api/notifications`) é encaminhado para o serviço correspondente, na rede interna do Docker [8];
- **Autenticação:** o JWT [6] é validado no gateway antes de o pedido chegar ao serviço. As rotas públicas do Auth (registo, login, renovação do token e recuperação de palavra-passe) são as únicas que não exigem token;

- **Limite de pedidos:** as rotas públicas do Auth aceitam no máximo 10 pedidos por minuto por endereço IP, para dificultar ataques de força bruta ao login;
- **CORS:** só a origem do *frontend* pode fazer pedidos a partir do browser;
- **Tratamento de falhas:** quando um serviço está em baixo ou não responde, o gateway devolve uma resposta clara;

O JWT [6] é validado no gateway e outra vez em cada serviço, com a mesma chave. Assim, um serviço nunca confia apenas na rede: mesmo que um pedido chegasse sem passar pelo gateway, seria recusado sem um token válido.

4.3 Microsserviços

4.3.1 Serviço de Autenticação (Auth)

Responsabilidade: contas de utilizador, autenticação, perfis e política de faltas.

- **Registo e login:** só os clientes se registam; as contas de gestor são criadas pelo administrador e associadas a um restaurante. O login devolve um *access token* de 15 minutos e um *refresh token* de 7 dias;
- **Tokens:** o *refresh token* é aleatório (64 bytes) e só o seu *hash* SHA-256 é guardado. Cada renovação revoga o token anterior (rotação);
- **Recuperação de palavra-passe:** a resposta é igual exista ou não o email, para não revelar que contas existem; a ligação de recuperação é enviada pelo serviço de notificações;
- **Perfil:** consultar e alterar nome, email, telefone, palavra-passe e fotografia;
- **Eliminação da conta (RGPD):** os dados pessoais são anonimizados e é publicado o evento `UserDeleted`, para que os outros serviços anonimizem também os seus;
- **Administração:** listar utilizadores, criar gestores, bloquear e desbloquear clientes, e definir o limite de faltas (por defeito 3);
- **Contagem de faltas:** consome `NoShowRegistered` (regra 4, Secção **??**).

Dados: `users`, `refresh_tokens`, `password_reset_tokens`, `counted_no_shows` e `policies`. **Endpoints:** 17.

4.3.2 Serviço de Restaurantes (Restaurants)

Responsabilidade: restaurantes, mesas, turnos, estado das mesas, tempo médio de refeição e estatísticas de ocupação.

- **Restaurantes:** pesquisa por texto, data e número de pessoas; detalhes com horários, turnos e capacidades das mesas; criação, edição e desativação pelo administrador;
- **Mesas:** criar, editar e remover mesas e capacidades (o gestor só no seu restaurante);
- **Ocupação:** cada refeição fica registada em `table_sessions` (hora em que o grupo se sentou e em que a mesa foi libertada). Libertar uma mesa publica `TableFreed`;
- **Tempo médio de refeição:** começa no valor por defeito (90 minutos) e passa a ser a média real (regra 5), ou um valor fixo definido pelo gestor (modo manual);
- **Estatísticas:** ocupação por dia e tempo médio real nos últimos N dias;
- **Cópia para a fila de espera:** sempre que o restaurante, as mesas ou o estado de uma mesa mudam, publica `RestaurantSnapshotChanged`.

Dados: `restaurants` (horários e turnos em `jsonb`), `tables`, `table_sessions` e `policies`. **Endpoints:** 16.

4.3.3 Serviço de Reservas (Reservations)

Responsabilidade: reservas, disponibilidade, chegada, não comparência e relatórios.

- **Disponibilidade:** para um dia e número de pessoas, devolve todas as horas dos turnos, indicando as que estão esgotadas;
- **Reservar:** valida o restaurante, o turno e o tamanho do grupo. Uma reserva repetida do mesmo cliente à mesma hora dá conflito; uma reserva cancelada não bloqueia uma nova reserva à mesma hora;
- **Alterar e cancelar:** o cliente só vê e altera as suas reservas (as de outros clientes respondem como inexistentes); o gestor pode cancelar as do seu restaurante;
- **Reservas manuais:** o gestor regista clientes sem conta (nome e telefone);
- **Chegada:** sugere a mesa livre mais pequena que serve o grupo e publica `ReservationArrived`; a refeição fecha-se com *complete*;
- **Não comparência:** publica `NoShowRegistered`;
- **Relatórios:** número de reservas e taxa de faltas por restaurante ou por região.

Os dados do restaurante (turnos, capacidades, tempo médio de refeição) são pedidos ao serviço de restaurantes por HTTP, reencaminhando o cabeçalho `Authorization` do pedido original.

Dados: `reservations`, com o histórico de estados em `jsonb` e o nome do cliente (`customer_name`) para o ecrã do gestor. **Endpoints:** 11.

4.3.4 Serviço de Fila de Espera (Waitlist)

Responsabilidade: fila de espera, tempo estimado e atribuição de mesas livres.

- **Fila:** o cliente entra na fila, consulta a posição, o total de grupos e o tempo estimado, e pode sair; o gestor consulta a fila do seu restaurante;
- **Estimativa:** Cada grupo à frente ocupa a mesa adequada que fica livre mais cedo durante o tempo médio de refeição; o tempo do novo grupo é o da primeira mesa adequada a ficar livre (função EstimateMinutes).
- **Tolerância:** um processo em segundo plano verifica a cada 30 segundos os grupos chamados que não se apresentaram dentro da tolerância (15 minutos por defeito), passa-os a *Expired* e volta a oferecer a mesa;
- **Gestão manual:** o gestor pode chamar o próximo grupo e marcá-lo como sentado.

Dados: `waitlist_entries`, `restaurant_snapshots` e `upcoming_reservations`. **Endpoints:** 7.

4.3.5 Serviço de Notificações (Notifications)

Responsabilidade: envio de emails, registo e lista das notificações de cada utilizador. É o serviço que integra o serviço externo da plataforma, o Mailtrap.

- **Consumidores:** reserva confirmada, alterada e cancelada; grupo chamado da fila; pedido de revisão de bloqueio (enviado ao administrador); cliente bloqueado; recuperação de palavra-passe; eliminação de conta (anonimiza o registo);
- **Modelos:** sete modelos de mensagem em português, editáveis pelo administrador;
- **Preferências:** cada utilizador pode desligar os avisos por email;
- **Envio pela API do Mailtrap::** os emails são enviados para uma caixa de testes (sandbox) do Mailtrap [15] através da sua API HTTP, por HTTPS (porta 443), com um token de acesso e o identificador da caixa definidos em variáveis de ambiente. A caixa recebe todos os emails sem os entregar aos destinatários reais, o que permite verificar o conteúdo de cada mensagem durante o desenvolvimento e a demonstração;
- **Registo e falhas:** cada notificação fica registada com o estado Sent (aceite pelo Mailtrap [15]), Failed (com a mensagem de erro) ou Skipped (utilizador com os emails desligados). Uma falha no envio não bloqueia o consumidor, e cada notificação guarda o identificador da mensagem que lhe deu origem, para que um evento repetido não envie o mesmo email duas vezes.

Dados: `notification_log`, `preferences` e `templates`. **Endpoints:** 4.

4.4 Comunicação entre Serviços

Cada serviço publica um evento depois de gravar a alteração na sua base de dados. Os eventos levam os dados de que os consumidores precisam (por exemplo o email do cliente), porque um serviço nunca acede à base de dados de outro.

Evento	Publicado por	Consumido por
`ReservationConfirmed`	Reservations	Notifications, Waitlist
`ReservationChanged`	Reservations	Notifications, Waitlist
`ReservationCancelled`	Reservations	Notifications, Waitlist
`ReservationArrived`	Reservations	Restaurants, Waitlist
`NoShowRegistered`	Reservations	Auth, Waitlist
`TableFreed`	Restaurants	Waitlist
`RestaurantSnapshotChanged`	Restaurants	Waitlist
`WaitlistPromoted`	Waitlist	Notifications, Restaurants
`BlockReviewRequested`	Auth	Notifications
`UserBlocked`	Auth	Notifications
`UserDeleted`	Auth	Reservations, Waitlist, Notifications
`PasswordResetRequested`	Auth	Notifications

Tabela 4.1: Eventos trocados entre serviços

Como a mesma mensagem pode ser entregue mais de uma vez, **todos os consumidores são idempotentes**: processar duas vezes a mesma mensagem tem o mesmo efeito que processá-la uma vez (por exemplo, a falta é registada numa tabela cuja chave primária é o identificador da reserva, e cada notificação guarda o identificador da mensagem que lhe deu origem). Em caso de erro, cada consumidor tenta outra vez até três vezes, com intervalos de 5 segundos. Cada serviço tem as suas próprias filas no RabbitMQ [7] (por exemplo `waitlist-table-freed` e `notifications-reservation-confirmed`), de modo que cada um recebe a sua cópia do evento.

4.5 Dados

Cada serviço tem a sua base de dados no mesmo servidor PostgreSQL [14] e nunca acede à de outro; entre serviços só existem identificadores (`uuid`), nunca chaves estrangeiras. O esquema de cada serviço é versionado com *migrations* do Entity Framework Core [20], aplicadas automaticamente quando o serviço arranca. As regras de integridade estão também na própria base de dados: restrições únicas (email do utilizador, nome da mesa por restaurante, reserva do mesmo cliente à mesma hora), restrições `CHECK` (capacidade e número de pessoas positivos) e índices parciais (por exemplo, só os grupos à espera). As datas são guardadas em UTC e os horários dos restaurantes em hora de Lisboa.

4.6 Segurança

- **Autenticação:** JWT [6] assinado com HS256 [6], validado no gateway e em cada serviço, com os perfis Cliente, Gestor e Administrador. O token do gestor indica o seu restaurante;
- **Autorização:** políticas por perfil e verificação "gestor do próprio restaurante" (um gestor que tenta alterar mesas ou reservas de outro restaurante recebe 403). Um cliente que pede dados de outro cliente recebe 404, para não revelar que existem;
- **Palavras-passe e tokens:** palavras-passe com BCrypt [21]; *refresh tokens* e tokens de recuperação guardados apenas como *hash*;
- **Respostas genéricas:** o login e a recuperação de palavra-passe não revelam que emails existem; um cliente bloqueado recebe a mesma mensagem que credenciais erradas;
- **Rede:** só o gateway publica uma porta; os serviços, o PostgreSQL [14] e o RabbitMQ [7] ficam numa rede interna, com utilizador e palavra-passe;
- **RGPD (RNF09):** consentimento guardado no registo; eliminar a conta anonimiza nome, email e telefone em todos os serviços e mantém as reservas apenas para estatística. [13]

4.7 Documentação das APIs

Cada serviço documenta a sua API em OpenAPI [9] (RNF06). Todos os *endpoints* declaram as respostas possíveis, incluindo os erros (400, 401, 403, 404, 409, 422 e 503), e o esquema de autenticação Bearer. Os documentos são gerados automaticamente em cada compilação para a pasta `docs/openapi` [9] (um ficheiro por serviço), sem ser preciso ter os serviços a correr.

5. Resiliência e Observabilidade

Numa arquitetura de microsserviços, a falha de uma peça é uma situação normal e não uma exceção. Este capítulo descreve como a plataforma se comporta quando um serviço, uma dependência, a base de dados ou o *broker* de mensagens falham, que respostas recebe a aplicação, e como se acompanha cada pedido de ponta a ponta.

5.1 Princípios

- **Isolamento:** cada serviço tem a sua base de dados e o seu processo; a falha de um serviço não impede os outros de responder;
- **Comunicação assíncrona:** os serviços avisam-se por eventos no RabbitMQ [7]. Se o consumidor estiver em baixo, o evento fica na fila e é processado quando o serviço volta. É o que garante o RNF04: uma falha no serviço de notificações não impede a criação de reservas;
- **Autenticação sem dependências:** o JWT [6] é validado localmente no gateway e em cada serviço; sem o serviço de autenticação deixa de ser possível iniciar sessão, mas os tokens já emitidos continuam válidos;
- **Falhar depressa e com clareza:** uma dependência em baixo dá uma resposta 503 com uma mensagem que explica o que falhou, em vez de um erro genérico ou de um pedido pendurado.

5.2 Respostas de Erro

Todas as respostas de erro seguem o formato *Problem Details* (RFC 9457) e estão em português, incluindo as que o ASP.NET Core [22, 23] gera automaticamente (falta de token, validação dos campos, JSON mal formado). Cada resposta indica o serviço que produziu o erro (`service`) e o identificador do pedido (`traceId`), que permite encontrá-lo nos registos e no Jaeger [24]. Por exemplo, com o serviço de reservas parado:

```
HTTP/1.1 503 Service Unavailable
X-Trace-Id: b289140cbcf9c7ea82409207641383cd
{
  "title": "Serviço indisponível",
  "status": 503,
```

```
  "detail": "O serviço de reservas está indisponível.",
  "service": "reservations",
  "traceId": "00-b289140cbcf9c7ea82409207641383cd-ed21d1c051186c8d-01"
}
```

Código	Quando
400	Dados inválidos, com a lista de erros por campo
401	Sem sessão iniciada: token em falta, inválido ou expirado
403	Perfil sem permissão, ou gestor a atuar noutro restaurante
404	Recurso inexistente (ou de outro cliente)
409	Conflito: reserva repetida, mesa já livre, sem mesas para a hora pedida
422	Pedido impossível: nenhuma mesa serve o grupo
429	Demasiados pedidos nas rotas públicas do Auth
503	Serviço, base de dados ou RabbitMQ [7] indisponíveis
504	Um serviço não respondeu a tempo ao gateway

Tabela 5.1: Códigos de erro devolvidos à aplicação

5.3 Resiliência na Comunicação Síncrona

A chamada do serviço de reservas ao serviço de restaurantes é a única dependência síncrona entre serviços, e por isso é protegida com três mecanismos:

- **Tempo limite:** cada tentativa tem no máximo 2 segundos (8 segundos no total);
- **Novas tentativas:** até duas, para falhas transitórias;
- Se pelo menos metade de quatro ou mais chamadas falhar num intervalo de 10 segundos, o circuito abre durante 15 segundos. Enquanto está aberto, os pedidos falham imediatamente, sem esperar por um serviço que se sabe estar em baixo, o que evita acumular pedidos pendurados e consumir recursos.

5.4 Comportamento em Caso de Falhas

Os cenários seguintes foram executados no ambiente completo (Docker Compose) [25], parando um container de cada vez. Estão automatizados numa *collection* do Postman [12] própria. Os tempos indicados foram medidos.

Peça parada	Resposta à aplicação	O que demonstra
Serviço de reservas	503 do gateway: "O serviço de reservas está indisponível." Os restantes serviços continuam a responder normalmente.	Isolamento entre serviços; o gateway responde com clareza em vez de um 502 sem corpo.
Serviço de autenticação	Login: 503. Pedidos com um token já emitido: 200.	O JWT [6] é validado localmente: o sistema continua a funcionar para quem já tem sessão.
Serviço de restaurantes	Reservar: 503 do serviço de reservas. 1.ª tentativa em cerca de 6,7 s (tentativas com tempo limite), 2.ª em cerca de 2 s, a partir da 3.ª em menos de 20 ms. "As minhas reservas" continua a funcionar.	Tempo limite e novas tentativas; só falham as operações que dependem do serviço em baixo.
Serviço de notificações	A reserva é criada (201). Ao voltar, o serviço processa o evento que ficou na fila e a notificação aparece.	Comunicação assíncrona e consistência eventual (RNF04).
PostgreSQL	503: "A base de dados do serviço de restaurantes está indisponível."	Falhas de infraestrutura distinguem-se de erros do pedido.
RabbitMQ	Reservar: 503 ao fim de 5 s: "a alteração foi gravada, mas os outros serviços não foram avisados". Ao voltar, os serviços religam-se sozinhos em 20 a 40 s.	Limitação conhecida (Secção 5.5).

Tabela 5.2: Comportamento do sistema com cada peça parada

5.5 Limitações Conhecidas

Cada serviço grava a alteração na sua base de dados e só depois publica o evento. Se o RabbitMQ [7] estiver em baixo nesse momento, a alteração fica gravada mas os outros serviços não são avisados: por exemplo, a reserva existe, mas a fila de espera e as notificações nunca recebem o evento de confirmação. O sistema deteta a situação e responde 503 ao fim de 5 segundos, em vez de deixar o pedido pendurado, mas a inconsistência mantém-se.

A solução é o padrão *transactional outbox*: o evento é gravado numa tabela da mesma base de dados, na mesma transação da alteração, e um processo em segundo plano publica-o quando o RabbitMQ [7] estiver disponível. O MassTransit oferece esta funcionalidade para o Entity Framework Core; fica como trabalho futuro, porque exige tabelas novas nos cinco serviços.

6. Testes

A estratégia de testes cobre três níveis (RNF08): testes unitários da lógica de decisão, testes de integração de cada serviço com uma base de dados real, e testes de sistema que percorrem os fluxos completos através do API Gateway. Todos os testes são automáticos e repetíveis.

6.1 Testes de Integração

Cada serviço tem testes de integração que arrancam a aplicação completa em memória (`WebApplicationFactory`) e enviam pedidos HTTP reais, com:

- **Base de dados real:** um PostgreSQL 17 [14] novo por classe de testes, criado num container pelo Testcontainers, com as *migrations* reais aplicadas. Os testes usam assim as mesmas restrições, índices e bloqueios que a produção;
- **Mensagens em memória:** o MassTransit é substituído pelo seu *test harness*, que permite publicar eventos e verificar quais foram publicados e consumidos;
- **Tokens de teste:** gerados com a mesma biblioteca e chave que o serviço de autenticação, para cada perfil;
- **Dependências simuladas:** o serviço de restaurantes é substituído nos testes do serviço de reservas, incluindo um modo em que simula estar em baixo.

6.2 Testes Obrigatórios

A análise definiu um conjunto de testes obrigatórios para as regras críticas. Todos estão implementados e passam:

Teste	Serviço
10 pedidos de reserva em paralelo para 3 mesas dão exatamente 3 reservas (3 respostas 201 e 7 respostas 409)	Reservations
`ShouldProtect`: mesa de 4 livre às 19:40 com reserva de 3 às 20:00 fica protegida	Waitlist
`ShouldProtect`: com a reserva às 21:30, a mesa vai para a fila	Waitlist
`ShouldProtect`: com duas mesas livres, uma vai para a fila	Waitlist
Um grupo de 6 não bloqueia um grupo de 2 quando só há mesa de 4	Waitlist
Duas mensagens `TableFreed` seguidas não promovem o mesmo grupo	Waitlist
A mesma falta é contada uma só vez	Auth
A média do tempo de refeição só substitui o valor inicial com 20 amostras e ignora refeições fora do intervalo de 15 minutos a 4 horas	Restaurants
Um gestor não consegue alterar mesas nem reservas de outro restaurante (403)	Restaurants, Reservations

Tabela 6.1: Testes obrigatórios

Para além destes, os testes de integração cobrem o registo, o login e a rotação de tokens, o bloqueio de clientes, a eliminação de conta (RGPD) [13], a disponibilidade com horas esgotadas, a chegada com sugestão de mesa, a idempotência das notificações, as preferências de email, a resposta 503 quando uma dependência está em baixo e as mensagens de erro em português.

6.3 Resultados

Projeto de testes	Testes
Auth	26
Restaurants	20
Reservations	29
Waitlist	26
Notifications	13
Total	**114**

Tabela 6.2: Testes unitários e de integração por serviço (todos aprovados)

Os testes executam-se com `dotnet test` (com o Docker [8] a correr, para os containers do PostgreSQL) [14] e terminam em cerca de 30 segundos. Durante o desenvolvimento foi detetado e corrigido um problema de estabilidade nos testes: o *test harness* do MassTransit é partilhado por todos os testes de uma classe e, depois do primeiro teste, a sua espera por mensagens respondia antes de a mensagem nova chegar, o que fazia falhar testes consoante a ordem de execução. As esperas passaram a consultar a lista de mensagens até um prazo, e a suíte foi executada várias vezes seguidas sem falhas.

6.4 Testes de Sistema

Os testes de sistema usam o ambiente completo (Docker Compose) [25] e passam pelo API Gateway, tal como a aplicação. Estão em duas *collections* do Postman [12], executáveis também na linha de comandos com o Newman:

- **Fluxo completo** (34 pedidos, 77 verificações): o administrador cria um restaurante, as mesas e o gestor; o cliente regista-se, renova o token, pesquisa, consulta a disponibilidade, reserva (e recebe 409 na reserva repetida) e entra na fila; o gestor marca a chegada (com sugestão de mesa), a falta, liberta a mesa e recebe 403 noutro restaurante; o administrador confirma a falta contada, bloqueia o cliente, e o login seguinte é recusado;
- **Cenários de falha** (64 verificações): os seis cenários da Tabela 5.2, executados pedido a pedido, parando e arrancando os containers indicados.

Ambas incluem verificações comuns a todos os pedidos: a resposta tem o cabeçalho `X-Trace-Id` e os erros vêm no formato *Problem Details*. Todas as verificações passaram.

7. Containers – Docker

7.1 Imagens

Todos os serviços usam o mesmo *Dockerfile*, em duas fases e com imagens oficiais da Microsoft: a compilação é feita na imagem do SDK do .NET 10 [22] e o resultado é copiado para a imagem `aspnet:10.0` [23], mais pequena e sem ferramentas de compilação. O processo corre com um utilizador sem privilégios de administrador. O PostgreSQL [14], o RabbitMQ [7] e o Jaeger [24] usam também as imagens oficiais.

7.2 Ambiente com Docker Compose

O ficheiro `deploy/docker-compose.yml` arranca o ambiente completo com um só comando:

```
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
```

Container	Função
`gateway`	API Gateway; único com porta publicada (8080)
`auth-api`, `restaurants-api`, `reservations-api`, `waitlist-api`, `notifications-api`	Microsserviços, apenas na rede interna
`postgres`	PostgreSQL 17 [14]; um *script* de arranque cria as cinco bases de dados e um utilizador só de leitura para exportação de dados
`rabbitmq`	*Broker* de mensagens
`jaeger`	Rastreio distribuído; interface apenas acessível na própria máquina

Tabela 7.1: Containers do ambiente

Os serviços só arrancam depois de o PostgreSQL [14] e o RabbitMQ [7] estarem prontos, e cada serviço aplica as suas *migrations* ao arrancar. As credenciais e a chave do JWT [6] vêm de um ficheiro `.env` que nunca é guardado no repositório; o repositório inclui apenas um `.env.example`.

8. Conclusão e Trabalho Futuro

8.1 Requisitos Cumpridos

A Tabela 8.1 relaciona cada requisito definido na Secção 2.3 com a sua implementação.

Req.	Estado	Implementação
RF01	Cumprido	Registo de clientes e login (Auth); gestores criados pelo administrador
RF02	Cumprido	Recuperação de palavra-passe com ligação por email (Auth e Notifications)
RF03	Cumprido	Pesquisa por texto (nome, cozinha, cidade), cozinha, data e número de pessoas; a hora escolhe-se na disponibilidade (Restaurants)
RF04	Cumprido	Detalhes do restaurante e horas disponíveis, incluindo as esgotadas (Restaurants e Reservations)
RF05	Cumprido	Reserva com data, hora e número de pessoas, sem exceder as mesas
RF06	Cumprido	Consultar (próximas e histórico), alterar e cancelar reservas
RF07	Cumprido	Entrar e sair da fila, posição, total de grupos e tempo estimado
RF08	Cumprido	Emails de reserva confirmada, alterada e cancelada e de mesa disponível para o grupo da fila (Notifications)
RF09	Cumprido	Reservas do restaurante, reservas manuais, chegada com sugestão de mesa e não comparência
RF10	Cumprido	Fila do restaurante, chamar o próximo grupo que cabe e atribuição automática de mesas livres
RF11	Cumprido	Mesas, capacidades, turnos, horários e tempo médio de refeição
RF12	Parcial	Ocupação por dia e tempo médio de refeição real (Restaurants); taxa de faltas nos relatórios do administrador. Faltam estatísticas dos tempos de espera da fila
RF13	Cumprido	Criar e editar restaurantes; remover é feito por desativação, para manter o histórico
RF14	Cumprido	Lista de utilizadores com filtros, bloqueio e desbloqueio; os clientes que atingem o limite de faltas ficam marcados para revisão

Req.	Estado	Implementação
RF15	Cumprido	Limite de faltas (Auth) e tempo de refeição, margem e tolerância por defeito (Restaurants)
RF16	Cumprido	Reservas e taxa de faltas por restaurante ou por região
RF17	Cumprido	Alterar palavra-passe e fotografia de perfil
RNF01	Por medir	Estimativa atualizada por eventos, em menos de um segundo após cada alteração; não foram feitos testes de carga para medir o percentil 95
RNF02	Parcial	JWT [6] validado no gateway e em cada serviço, BCrypt [21], controlo de acesso por perfil. O ambiente local usa HTTP; o HTTPS fica a cargo do ponto de entrada em produção
RNF03	Cumprido	Só o gateway publica uma porta; serviços, base de dados e RabbitMQ [7] na rede interna
RNF04	Cumprido	Comunicação assíncrona; demonstrado com o serviço de notificações parado (Tabela 5.2)
RNF05	Cumprido	Os microsserviços devem ser executados em containers Docker [8] e não guardar estado em memória: os dados ficam na respetiva base de dados PostgreSQL [14] e as mensagens no RabbitMQ [7].
RNF06	Cumprido	Um serviço e uma base de dados por área de negócio, *migrations* versionadas e OpenAPI [9] de cada serviço
RNF07	Protótipo	Abordagem *mobile first* definida no protótipo de alta fidelidade; a aplicação cliente não fez parte da implementação
RNF08	Cumprido	114 testes unitários e de integração e duas *collections* do Postman [12] (Capítulo 6)
RNF09	Cumprido	Consentimento no registo e eliminação de conta com anonimização em todos os serviços

Tabela 8.1: Rastreabilidade dos requisitos

8.2 Conclusão

O GrabATable foi implementado como um conjunto de cinco microsserviços independentes atrás de um API Gateway, cada um com a sua base de dados, comunicando por eventos através do RabbitMQ [7]. A principal inovação identificada na análise, a gestão conjunta das reservas e da fila de espera, foi concretizada sem transações distribuídas: cada decisão crítica (não exceder as mesas, contar uma falta uma só vez, não chamar o mesmo grupo duas vezes) é tomada dentro de uma única base de dados, e a informação de que a fila de espera precisa chega-lhe por eventos.

A arquitetura foi validada não só no caminho feliz, mas também perante falhas: com qualquer serviço, a base de dados ou o *broker* em baixo, o sistema responde de forma previsível e explica à aplicação o que falhou, e o rastreio distribuído permite seguir cada pedido através de todos os serviços. O comportamento está coberto por 114 testes automáticos e por testes de sistema que percorrem os fluxos completos e os cenários de

falha.

8.3 Trabalho Futuro

- ***Transactional outbox*:** garantir que nenhum evento se perde quando o RabbitMQ [7] está em baixo;
- **Aplicação cliente:** implementar a aplicação a partir do protótipo, com notificações em tempo real (por exemplo com SignalR) para a posição na fila;
- **Estatísticas da fila:** tempos de espera reais face aos estimados, para completar o RF12 e afinar a estimativa;
- **Desempenho:** testes de carga para verificar o RNF01 e calibrar o *HorizontalPodAutoscaler*;
- **Segurança em produção:** HTTPS no ponto de entrada e gestão de segredos num cofre;
- **Publicação numa *cloud*:** publicar o projeta na cloud.

Bibliografia

[1] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2ª ed. O'Reilly Media, 2021.

[2] James Lewis e Martin Fowler. *Microservices: a definition of this new architectural term*. 2014. URL: https://martinfowler.com/articles/microservices.html (acedido em 03/11/2026).

[3] Coolors. *Coolors Palette Generator*. URL: https://coolors.co (acedido em 03/11/2026).

[4] Google. *Google Fonts: Poppins e Inter*. URL: https://fonts.google.com (acedido em 03/11/2026).

[5] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. URL: https://www.w3.org/TR/WCAG21/ (acedido em 03/11/2026).

[6] Michael Jones, John Bradley e Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF), 2015. URL: https://www.rfc-editor.org/rfc/rfc7519.

[7] Broadcom. *RabbitMQ Documentation*. URL: https://www.rabbitmq.com/docs (acedido em 03/11/2026).

[8] Docker, Inc. *Docker Documentation*. URL: https://docs.docker.com/ (acedido em 03/11/2026).

[9] OpenAPI Initiative. *OpenAPI Specification*. URL: https://spec.openapis.org/oas/latest.html (acedido em 03/11/2026).

[10] Luke Wroblewski. *Mobile First*. Nova Iorque: A Book Apart, 2011.

[11] Jakob Nielsen. *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. URL: https://www.nngroup.com/articles/ten-usability-heuristics/ (acedido em 03/11/2026).

[12] Postman, Inc. *Postman Learning Center (Collections e Newman)*. URL: https://learning.postman.com/docs/ (acedido em 03/11/2026).

[13] Parlamento Europeu e Conselho da União Europeia. *Regulamento (UE) 2016/679 (Regulamento Geral sobre a Proteção de Dados)*. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (acedido em 03/11/2026).

[14] PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. URL: https://www.postgresql.org/docs/17/ (acedido em 03/11/2026).

[15] Mailtrap. *Mailtrap Documentation*. URL: https://docs.mailtrap.io/ (acedido em 03/11/2026).

[16] Jeffrey Stedfast. *MailKit: A cross-platform .NET mail-client library*. URL: https://github.com/jstedfast/MailKit (acedido em 03/11/2026).

[17] Object Management Group. *Business Process Model and Notation (BPMN), Version 2.0*. Especificação formal/2011-01-03. Object Management Group, 2011. URL: https://www.omg.org/spec/BPMN/2.0/.

[18] Figma, Inc. *Figma Design Documentation*. URL: https://help.figma.com (acedido em 03/11/2026).

[19] MassTransit. *MassTransit Documentation*. URL: https://masstransit.io (acedido em 03/11/2026).

[20] Microsoft. *Entity Framework Core Documentation*. URL: https://learn.microsoft.com/ef/core/ (acedido em 03/11/2026).

[21] Niels Provos e David Mazières. "A Future-Adaptable Password Scheme". Em: *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. 1999, pp. 81–91.

[22] Microsoft. *.NET Documentation*. URL: https://learn.microsoft.com/dotnet/ (acedido em 03/11/2026).

[23] Microsoft. *ASP.NET Core Documentation*. URL: https://learn.microsoft.com/aspnet/core/ (acedido em 03/11/2026).

[24] Jaeger Authors. *Jaeger Documentation*. URL: https://www.jaegertracing.io/docs/ (acedido em 03/11/2026).

[25] Docker, Inc. *Docker Compose Documentation*. URL: https://docs.docker.com/compose/ (acedido em 03/11/2026).